LLM-Driven Navigation Agent in a Webots 3D World

Diksha Jangam

May 31, 2026

Contents

1. Introduction

This document describes a complete intelligent-agent system that navigates a 3D Webots environment using:

- Large Language Model (LLM) reasoning for goal selection,
- Multi-path candidate generation and LLM judging,
- Receding-horizon re-planning at each goal transition,
- A* path planning on an occupancy grid,
- A waypoint-following controller,
- Automatic visualization of the world and path,
- CSV logging of the agent's behaviour.

The agent receives a natural-language instruction such as:

> "Go to the yellow goal, then the red goal, then the green goal."

It interprets the instruction, selects an ordered sequence of goals, plans a collision-free path, and executes it in Webots.

2. System Overview

2.1 High-level pipeline

The system follows this pipeline:

1. User instruction is defined in the controller code.
2. LLM multi-path reasoning generates several candidate goal sequences.
3. A judge LLM scores and selects the best candidate.
4. An occupancy grid is built from static obstacles.
5. A* path planning computes a path for each goal segment.
6. A waypoint controller drives the robot along the path.
7. CSV logging records behaviour; visualization is generated at the end.

2.2 World and objects

The Webots world contains:

- A differential-drive robot with GPS and IMU,
- Three named goals (red, green, yellow) with fixed (x, y) coordinates,
- Rectangular static obstacles.

The goals are stored as:

```
OBJECTS = [
  {"name": "red goal",    "x": 0.19,  "y": 1.24},
  {"name": "green goal",  "x": 1.15,  "y": -1.26},
  {"name": "yellow goal", "x": -2.25, "y": 0.48}
]
```

3. Design Choices

3.1 LLM multi-path reasoning

Instead of a single LLM call, the system:

- Generates multiple candidate goal sequences using different style constraints (shortest distance, clockwise order, safety, simplicity),
- Uses a separate "judge" LLM to score each candidate on:
  - Faithfulness to the user instruction,
  - Route reasonableness,
  - Efficiency,
  - Clarity of coordinates.
- Selects the candidate with the highest score.

This improves robustness and reduces the chance of a single poor LLM output.

3.2 Receding-Horizon LLM Re-Planning

A major enhancement added to the system is **receding-horizon LLM planning**. Instead of deciding the full route once at the beginning, the agent:

1. Moves to the next goal,
2. Re-observes its current position,
3. Updates the list of remaining goals,
4. Calls the LLM again with updated context,
5. Selects the next goal dynamically.

This enables:

- Adaptation to unexpected behaviour,
- More robust decision-making,
- Local optimization at each step,
- Reduced sensitivity to initial LLM errors.

This turns the agent into a **closed-loop intelligent planner** rather than a one-shot planner.

3.3 A* path planning on an occupancy grid

A grid-based occupancy map is built around the robot:

- Obstacles are inflated by a safety margin,
- Each grid cell is marked as free or occupied,
- A* search finds a path between start and goal cells.

A* is chosen for its determinism and suitability for static environments.

3.4 Waypoint-based controller

The continuous path is discretized into waypoints. For each waypoint:

- Distance and heading error are computed using GPS and IMU,
- A rule-based controller selects one of:
  - GO_FORWARD,
  - TURN_LEFT,
  - TURN_RIGHT,
  - STOP.

  A speed profile reduces forward speed near the goal.

3.5 Discrete action space

The action space is intentionally discrete for stability and interpretability.

3.6 CSV logging

Each control step logs:

- (x, y) position,
- Yaw,
- Waypoint index,
- Action,
- Distance to waypoint,
- Heading error.

3.7 Visualization

A Matplotlib visualization is generated at the end:

- Obstacles as gray rectangles,
- Goals with correct colors,
- Start position,
- Executed path as a polyline.

4. Implementation Details

4.1 LLM configuration

The system uses a Groq-hosted LLM via an OpenAI-compatible API:

- API key from `OPENAI_API_KEY`,
- Main model for candidate generation,
- Judge model for scoring.

The multi-path function:

```
best_goals, meta = solve_goals_with_multipath(
    user_instruction, OBJECTS, current_pos, n_candidates=4
)
```

returns the best goal sequence and metadata.

4.2 Snapping LLM Output to Real Goals

LLMs sometimes hallucinate coordinates. To prevent unreachable or invalid goals, the system now:

1. Takes the first LLM-generated coordinate,
2. Computes its distance to each remaining real goal,
3. Snaps it to the nearest real goal,
4. Ensures A* always receives a valid target.

This guarantees:

- No hallucinated coordinates,
- No unreachable goals,
- No A* failures,
- Correct behaviour even with imperfect LLM output.

4.3 Occupancy grid and A*

Grid resolution:

```
GRID_RES = 0.20
GRID_RADIUS = 10
```

Obstacles are inflated:

```
OBSTACLE_INFLATION = 0.15
```

A* converts world coordinates to grid indices, searches, and converts back.

4.4 Goal stop radius

To avoid colliding with the goal object:

```
GOAL_STOP_RADIUS = 0.30
```

The planner targets a "safe" point before the goal.

4.5 Waypoint following

At each step:

1. Select current waypoint,
2. Compute distance and heading error,
3. If close enough, mark waypoint reached,
4. Otherwise choose an action.

4.6 Goal completion detection

When the final waypoint of a segment is reached, the system prints:

```
[GOAL] <goal name> reached successfully.
```

4.7 CSV logging

A CSV file `log.csv` is created in the project root.

4.8 Visualization

The function `visualize_world_and_paths()` draws obstacles, goals, and the executed path.

5. Running the System

5.1 Dependencies

```
pip install matplotlib requests
```

5.2 Webots setup

1. Open Webots.
2. Load `plane_env.wbt`.
3. Set the robot controller to `controller_plane_env`.

5.3 Environment variable

```
export OPENAI_API_KEY=your_key_here   # Linux/macOS
set OPENAI_API_KEY=your_key_here      # Windows
```

5.4 Changing the route

Modify the instruction in the controller:

```
user_instruction = "Choose the most efficient route to visit all goals."
```

5.5 Outputs

- `log.csv` – behaviour log,
- `map.png` – visualization,
- Console – LLM scores, waypoint progress, goal completion.

6. Example Natural-Language Instructions

- "Choose the most efficient route to visit all goals."
- "Visit all goals in the safest order."
- "Pick the best order to visit the goals."
- "Avoid tight spaces and choose a safe route."
- "Visit the goals in a clockwise order."
- "Go to the red goal, then the yellow goal, then the green goal."

7. Example Logs and Behaviour

7.1 Console output

Typical output:

```
[LLM] Generating multi-path goal candidates...
[LLM] Winner index: 0
[SYSTEM] Waypoint 14 reached.
[GOAL] red goal reached successfully.
[SYSTEM] All waypoints completed.
Saved visualization to: map.png
```

7.2 CSV snippet

```
-1.80, -0.70, 1.57, 12, GO_FORWARD, 0.35, 0.10
```

8. Discussion and Limitations

8.1 Strengths

- Clear separation between high-level reasoning and low-level control,
- Deterministic planning via A*,
- Transparent behaviour via logging and visualization,
- Natural-language route specification,
- Receding-horizon LLM re-planning improves robustness.

8.2 Limitations

- Static obstacles only,
- Simple discrete controller,
- LLM quality affects behaviour,
- Multi-path LLM calls can hit rate limits.

9. Conclusion

This system demonstrates a complete intelligent-agent pipeline in a Webots 3D world:

- LLM-based goal interpretation,
- Multi-path candidate generation and judging,
- NEW: Receding-horizon re-planning at each goal,
- A* path planning on an occupancy grid,
- Waypoint following with a discrete controller,
- Behaviour logging and visualization.

It satisfies the challenge requirements for:

- A working agent in a virtual world,
- Clear observation and action formats,
- Example inputs and outputs,
- Documented design choices.